[1] ✓ 1s

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[2] ✓ 7s

```
from google.colab import files
uploaded = files.upload()

data = pd.read_csv("Credit_Card_Default_500_Records.csv")
data.head()
```

Choose Files Credit_Car...Records.csv

Credit_Card_Default_500_Records.csv(text/csv) - 11500 bytes, last modified: 7/6/2026 - 100% done

Saving Credit_Card_Default_500_Records.csv to Credit_Card_Default_500_Records (1).csv

	Income	Age	Loan	Loan to Income	Default
0	103810	28	2639	0.025	0
1	117196	38	17049	0.145	0
2	49256	29	49265	1.000	0
3	33434	64	49540	1.482	1
4	91482	26	39698	0.434	0

[3]
✓ 0s

```
print(data.shape)
print(data.info())
print(data.describe())
print(data.isnull().sum())
```

```
(500, 5)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Income          500 non-null    int64  
 1   Age             500 non-null    int64  
 2   Loan            500 non-null    int64  
 3   Loan to Income  500 non-null    float64
 4   Default         500 non-null    int64  
dtypes: float64(1), int64(4)
memory usage: 19.7 KB
None
              Income         Age          Loan  Loan to Income     Default
count     500.00000  500.000000    500.000000      500.000000  500.000000
mean    73154.52200   42.888000  25577.642000        0.433700    0.164000
std     28816.88444   12.870071  14250.329485        0.370582    0.370646
min     20425.00000   21.000000   1026.000000        0.013000    0.000000
25%     49899.00000   32.000000  13240.250000        0.183000    0.000000
50%     73415.00000   42.000000  26259.000000        0.338000    0.000000
75%     99212.50000   54.000000  37767.250000        0.566000    0.000000
max    119943.00000   65.000000  49986.000000        2.243000    1.000000
Income            0
Age               0
Loan              0
Loan to Income    0
```

[4]
✓ 0s

```
X = data[['Income', 'Age', 'Loan', 'Loan to Income']]
y = data['Default']

print(X.head())
print(y.head())
```

```
   Income  Age   Loan  Loan to Income
0  103810   28   2639           0.025
1  117196   38  17049           0.145
2   49256   29  49265           1.000
3   33434   64  49540           1.482
4   91482   26  39698           0.434
0    0
1    0
2    0
3    1
4    0
Name: Default, dtype: int64
```

[5]
✓ 0s

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(400, 4)
(100, 4)
```

[6]
✓ 0s

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

LogisticRegression ❶ ❷

```
LogisticRegression(max_iter=1000)
```

[7]
✓ 0s

```
y_pred = model.predict(X_test)

result = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred
})

print(result.head(10))
```

```
   Actual  Predicted
0       0          0
1       0          0
2       0          0
3       1          0
4       0          0
5       0          0
6       0          0
7       0          0
8       0          0
9       1          1
```

[8]
✓ 0s

```
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.92

Confusion Matrix:
[[83  1]
 [ 7  9]]

Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.99      0.95        84
           1       0.90      0.56      0.69        16

    accuracy                           0.92       100
   macro avg       0.91      0.78      0.82       100
weighted avg       0.92      0.92      0.91       100
```